The Job Sequencing and Tool Switching Problem

Part VIII: Branch-and-Benders Decomposition (BBC)

A master sequence problem with a polynomial KTNS subproblem

Scope and status. This note develops the Branch-and-Benders-cut (BBC) exact method for the uniform SSP. The SSP splits cleanly into a *sequencing* decision and a *tooling* decision: fix the job sequence and the optimal tooling cost is computed in polynomial time by KTNS [Tang and Denardo(1988)]. BBC puts the sequence in a *master* ATSP (with a surrogate θ for the tool cost) and the tooling in a *subproblem* solved by KTNS, linked by Benders optimality cuts. The decisive structural feature — and the reason BBC is attractive here — is that **the subproblem is polynomial**, in contrast to the Dantzig–Wolfe/column-generation route of Part II whose pricing subproblem is itself an NP-hard SSP. The integer (lazy) version is implemented and exact (`src/BBC/branch_and_benders_cut_cplex.py`); separating Benders cuts at *fractional* master solutions is an open direction (section 6).

Contents

The Decomposition

Notation. Jobs $J = \{1, \dots, n\}$, tools T, capacity b, required sets $T_j \subseteq T$. A dummy job 0 with $T_0 = \emptyset$ acts as a free start/end depot, so the schedule is an open path over the real jobs (no tool cost is charged for loading the first configuration). For a job sequence σ, $\mathrm{KTNS}(\sigma)$ denotes the minimum number of tool switches over σ, computed by the Keep-Tool-Needed-Soonest policy.

The split. A feasible SSP solution is a pair (sequence σ, tooling). For a *fixed* σ the optimal tooling is decided independently and exactly:

> **Theorem 1.1** (KTNS solves the subproblem exactly and in polynomial time [Tang and Denardo(1988)])**.** *For any fixed job sequence σ, the minimum number of tool switches equals* $\mathrm{KTNS}(\sigma)$*, computable in $O(n\,|T|)$ time.*

Hence

$$Z^*_{\mathrm{SSP}} \;=\; \min_{\sigma}\; \mathrm{KTNS}(\sigma),$$

the minimisation being over the $n!$ job sequences (a permutation problem). BBC optimises this by branch-and-cut on a master that chooses σ, deferring the evaluation $\mathrm{KTNS}(\sigma)$ to a callback.

Remark 1.2 (Why this split is the right one for the SSP)**.** Two decompositions compete (cf. Parts II and IV). *Dantzig–Wolfe / column generation* (Part II) generates configurations as columns; its pricing subproblem is a set-union knapsack and, for whole sequences, an NP-hard SSP. *Benders* fixes the complicating variables (the sequence) and leaves a subproblem that here is *polynomial* (KTNS). Benders therefore guarantees that exact cuts can always be produced cheaply — the property the CG route lacks. The price is that the master's lower bound is only as strong as the cuts accumulated so far.

The Master Problem

The master is an asymmetric TSP over $J \cup \{0\}$ with arc variables $x_{ij} \in \{0, 1\}$ ($x_{ij} = 1$ iff job j immediately follows job i) and a single continuous surrogate $\theta \geq 0$ standing for the tool-switching cost of the chosen sequence.

BBC master (relaxed; cuts added dynamically)

$$\min \quad \theta \tag{1}$$

$$\text{s.t.} \quad \sum_{j \neq i} x_{ij} = 1, \quad \sum_{i \neq j} x_{ij} = 1 \qquad \forall\, i, j \in J \cup \{0\} \tag{2}$$

$$\text{(subtour elimination on } x\text{)} \qquad \text{(lazy SEC or MTZ, remark 2.1)} \tag{3}$$

$$\theta \;\geq\; \sum_{i \neq j} w_{ij}\, x_{ij}, \qquad w_{ij} = \max\bigl(0,\; |T_i \cup T_j| - b\bigr) \tag{4}$$

$$\theta \;\geq\; \text{(Benders optimality cuts, section 3)} \tag{5}$$

$$x_{ij} \in \{0, 1\}, \quad \theta \geq 0.$$

Constraints (2)–(3) make x a Hamiltonian cycle through the depot, i.e. an open job sequence $\sigma(x)$. Inequality (4) is the *pairwise lower bound*: each executed transition $i \to j$ forces at least $w_{ij} = \max(0, |T_i \cup T_j| - b)$ switches, because the two jobs' tools cannot both fit unless $|T_i \cup T_j| \leq$

b [Tang and Denardo(1988), Laporte et al.(2004)Laporte, Salazar-González, and Semet]. It is valid *a priori* and seeds the master with a non-trivial bound before any Benders cut is generated.

Remark 2.1 (Subtour elimination in the master)**.** Any TSP subtour scheme works for (3): lazy DFJ cuts (separated on integer incumbents) or the polynomial MTZ constraints of Part IV (`04_mtz_formulation.tex`). Note the roles differ from Part IV: there MTZ replaced *configuration*-graph subtour constraints in a non-compact model; here it would only order the n *jobs*, where it is unproblematic and exact.

Benders Optimality Cuts

The subproblem has no linear-programming dual (KTNS is a combinatorial algorithm), so the cuts are *logic-based* Benders cuts [Vanderbeck and Wolsey(2010)]. Given an integer master incumbent with sequence σ, arc set $A(\sigma) = \{(i,j) : x_{ij} = 1\}$, evaluate $q = \mathrm{KTNS}(\sigma)$. If $\theta < q$ the incumbent is infeasible for the true cost and we add:

Proposition 3.1 (Validity of the combinatorial optimality cut)**.** *For the sequence σ with $q = \mathrm{KTNS}(\sigma)$ and $|A(\sigma)| = n$ (a depot cycle), the inequality*

$$\theta \;\geq\; q\Big(1 - \sum_{(i,j)\in A(\sigma)} \big(1 - x_{ij}\big)\Big) \;=\; q\Big(\textstyle\sum_{(i,j)\in A(\sigma)} x_{ij} - (n-1)\Big)$$

is valid: it forces $\theta \geq q$ whenever the master reproduces σ exactly, and is slack (≤ 0 on the right) for every other sequence.

Proof. If x reproduces σ then $\sum_{(i,j)\in A(\sigma)} x_{ij} = n$, the right-hand side is q, and $\theta \geq q = \mathrm{KTNS}(\sigma)$ is exactly the subproblem optimum (theorem 1.1), hence valid. If x omits at least one arc of σ then $\sum_{(i,j)\in A(\sigma)} x_{ij} \leq n-1$, the right-hand side is $\leq 0 \leq \theta$, so the cut imposes nothing. No feasible sequence is wrongly excluded. □

These "no-good" cuts are valid but weak. Each cut binds only on the single sequence that generated it, so naively BBC could enumerate exponentially many sequences. Two strengthenings are used/possible: (i) the always-on pairwise bound (4) keeps the root bound non-trivial; (ii) *cut lifting* — because $\mathrm{KTNS}(\sigma)$ depends only on the relative order in which tools are first/last needed (its 0-block structure, cf. Part I), a single evaluation certifies the same bound for the whole family of sequences sharing that structure, yielding cuts of the form $\theta \geq q\,(\sum_{(i,j)\in S} x_{ij} - |S| + 1)$ for a *subset* $S \subseteq A(\sigma)$ of order-critical arcs. Identifying minimal such S is the practical lever for BBC performance.

The Algorithm

BBC is a single branch-and-bound tree on the master with a lazy-constraint callback:

Branch-and-Benders-cut for the uniform SSP (integer cuts).

1. Build the master (1)–(4); install a lazy-constraint callback.
2. At each integer incumbent (x, θ):
   (a) extract the sequence $\sigma \leftarrow \sigma(x)$; if x contains a subtour, add a subtour cut and reject;

(b) compute $q \leftarrow \mathrm{KTNS}(\sigma)$ ($O(n|T|)$, exact by theorem 1.1);
(c) if $\theta < q$, add the optimality cut of proposition 3.1 (lifted if possible) and reject the incumbent;
(d) otherwise accept it — a genuine optimal-tooling schedule of cost $q = \theta$.

3. Continue until the tree is exhausted; return the incumbent, of cost Z^*_{SSP}.

Theorem 4.1 (Exactness and finiteness)**.** *The integer-cut BBC of section 4 terminates and returns* Z^*_{SSP}.

Proof. Every accepted incumbent satisfies $\theta = q = \mathrm{KTNS}(\sigma)$, hence corresponds to a real schedule of cost θ; thus the returned value is $\geq Z^*_{\mathrm{SSP}}$. Conversely the cuts of proposition 3.1 never remove a sequence's true cost (they are valid, proposition 3.1), so the master remains a relaxation and its optimum is $\leq Z^*_{\mathrm{SSP}}$. Finiteness: there are finitely many sequences; each rejected incumbent adds a cut that forbids re-accepting that sequence with too small a θ, so no sequence is rejected twice for the same reason. □

Position Among the Exact Methods

Method	Subproblem	Subproblem cost	
Compact ILP (Part I)	—	—	yes
DW / column generation (Part II)	pricing (set-union knapsack / SSP)	**NP-hard**	
Cluster-MTZ (Part IV)	—	—	*conditiona*
Branch-and-Benders (this Part)	tooling via KTNS	**poly** $O(n\|T\|)$	

BBC is the only route here whose subproblem is provably polynomial and whose exactness is unconditional. Its weakness is dual: the logic-based cuts can be weak, so the master bound may improve slowly without lifting. This is the mirror image of Part IV (cheap polynomial constraints, but a conditional/heuristic model) and of Part II (a strong LP bound, but an NP-hard subproblem).

Open Direction: Fractional Benders Cuts

OP (fractional Benders cuts; cf. `09_open_problems.tex`, OP 8). The implemented BBC separates cuts only at *integer* incumbents, so the LP relaxation at a node is bounded only by the pairwise inequality (4) plus previously added integer cuts. Develop a separation that produces a *valid* cut at a *fractional* master point $\bar{x}$ — e.g. from a relaxation of the KTNS recursion (a min-cost-flow / interval-matrix lower bound on the tooling cost as a function of fractional arc usage), or from the totally unimodular tooling LP of Catanzaro et al. for fixed-support sequences. Such user cuts would tighten the bound *before* branching and are the most promising avenue for scaling BBC beyond the sizes the integer-cut version reaches. Whether a polynomially separable family of fractional Benders cuts exists for the KTNS subproblem is open.

Remark 6.1 (Relation to the gap analysis)**.** By the grouping-exactness theorem of Part V, $Z^*_{\mathrm{SSP}} = \min_{\mathcal{G}} \mathrm{GTSP}(\mathcal{G})$; BBC reaches the same optimum from the sequence side. A primal heuristic for the BBC incumbent is therefore the JGP+GSP schedule of Part V, and the bound

$\theta \geq \max(K^* - 1, |U| - b)$ (Part V) may be added to (4) as an extra valid inequality to strengthen the root.

Implementation Notes

The CPLEX implementation is `src/BBC/branch_and_benders_cut_cplex.py` (public API in `branch_and_benders_cut.py`; shared subtour/sequence/KTNS utilities in `bbc_common.py`). The KTNS evaluation and subtour detection are in the lazy callback; the dummy depot gives the free initial load, so the reported objective is the number of inter-job switches (consistent with the convention of Parts I and V). Comparison baselines (LSS, SSPMF) are in `lss_formulation.py` and `sspmf_formulation.py`.

References

[Laporte et al.(2004)Laporte, Salazar-González, and Semet] G. Laporte, J. J. Salazar-González, and F. Semet. Exact algorithms for the job sequencing and tool switching problem. *IIE Transactions*, 36(1):37–45, 2004.

[Tang and Denardo(1988)] C. S. Tang and E. V. Denardo. Models arising from a flexible manufacturing machine, part i: Minimization of the number of tool switches. *Operations Research*, 36(5):767–777, 1988.

[Vanderbeck and Wolsey(2010)] F. Vanderbeck and L. A. Wolsey. Reformulation and decomposition of integer programs. In M. Jünger et al., editors, *50 Years of Integer Programming 1958–2008*, pages 431–502. Springer, Berlin, Heidelberg, 2010.